

Université de Lorraine – Master 1 Informatique
Année universitaire 2025-2026

RAPPORT DE PROJET

Intelligence Artificielle pour le jeu d'Awalé

*Conception d'un moteur de jeu compétitif
par recherche arborescente et apprentissage*

Naël BENZAADI & Hugo MORLET

Date de rendu : 3 mars 2026

Résumé

Ce rapport présente la conception et le développement d’une intelligence artificielle pour le jeu d’Awalé, un jeu de semailles traditionnel à information parfaite. Notre approche repose sur un moteur de recherche arborescente de type *Principal Variation Search* (PVS), enrichi de multiples techniques d’élagage issues des moteurs d’échecs modernes : *Reverse Futility Pruning*, *Razoring*, *Late Move Reductions* (LMR) et *Late Move Pruning*. La fonction d’évaluation, composée de 14 caractéristiques pondérées, est optimisée par un Algorithme Génétique durant une phase d’apprentissage de 55 minutes. Le moteur intègre également une *Endgame Tablebase* par analyse rétrograde, un *Opening Book* par auto-jeu profond, et une architecture zéro-allocation garantissant l’absence de pauses du ramasse-miettes Java.

Mots-clés

Intelligence Artificielle, Awalé, Minimax, Alpha-Beta, Principal Variation Search, Algorithme Génétique, Table de Transposition, Endgame Tablebase, K-NN

Table des matières

1	Introduction	1
2	Règles du jeu d'Awalé	1
2.1	Plateau et disposition initiale	1
2.2	Règles	1
3	Représentation et architecture	2
3.1	Représentation du plateau : FastBoard	2
3.2	Table de Transposition	2
4	Algorithme de recherche	3
4.1	Du Minimax au Principal Variation Search	3
4.2	Approfondissement itératif et fenêtres d'aspiration	3
4.3	Techniques d'élagage	3
4.4	Late Move Reductions (LMR)	3
4.5	Ordonnancement des coups	4
5	Fonction d'évaluation	4
5.1	Évaluation linéaire à 14 caractéristiques	4
5.2	Initialisation statistique par T-Statistic	5
5.3	K-Nearest Neighbors pour l'ordonnancement	5
6	Apprentissage et optimisation	5
6.1	Chargement et initialisation (< 1 min)	5
6.2	Algorithme Génétique (10 minutes)	5
6.3	Endgame Tablebase (15 minutes)	6
6.4	Opening Book et condensation (~30 minutes)	6
7	Résultats expérimentaux	6
8	Conclusion	7

1 Introduction

Ce rapport présente la conception et le développement d’une intelligence artificielle compétitive pour le jeu d’Awalé, réalisée dans le cadre du cours d’Intelligence Artificielle de Master 1 Informatique à l’Université de Lorraine (2025–2026).

Objectif. Le projet consiste à concevoir un joueur artificiel (*bot*) capable de remporter un tournoi face aux IA développées par les autres équipes. Le bot dispose d’une phase d’apprentissage d’une heure avant le championnat, puis doit prendre ses décisions en moins de 100 ms par coup en moyenne, sans calcul distribué. Une base de données de 303 situations de jeu issues de parties entre un joueur expérimenté et un novice est fournie comme ressource d’entraînement.

Approche. Notre développement a suivi une démarche itérative : partant d’algorithmes classiques de théorie des jeux (Minimax, Alpha-Beta), nous avons progressivement intégré des techniques issues des moteurs d’échecs modernes — *Principal Variation Search*, élagages agressifs, *Late Move Reductions* — tout en repensant l’architecture mémoire pour éliminer les pauses du ramasse-miettes Java. La fonction d’évaluation, composée de 14 caractéristiques du plateau, est optimisée automatiquement par un Algorithme Génétique. Le moteur final exploite l’intégralité des 55 minutes d’apprentissage pour calibrer ses poids, construire une base de données de fins de partie (*Endgame Tablebase*) et pré-calculer un répertoire d’ouvertures (*Opening Book*).

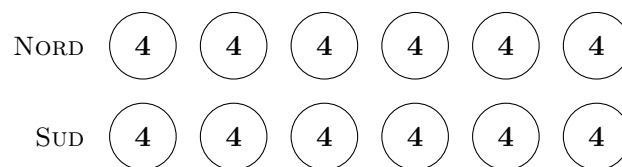
Plan. La section 2 rappelle les règles de l’Awalé. La section 3 décrit l’architecture interne du moteur. Les sections 4 et 5 détaillent respectivement l’algorithme de recherche et la fonction d’évaluation. La section 6 présente le pipeline d’apprentissage complet. Enfin, la section 7 analyse les performances obtenues en tournoi.

2 Règles du jeu d’Awalé

L’Awalé est un jeu de semailles (*Mancala*) traditionnel d’Afrique de l’Ouest, à information parfaite et à somme nulle. Sa complexité combinatoire a été étudiée par Romein et Bal [6], qui ont dénombré 889×10^9 positions légales.

2.1 Plateau et disposition initiale

Le plateau est constitué de deux rangées de 6 trous, chaque rangée appartenant à un joueur. En début de partie, chaque trou contient 4 graines, soit 48 graines au total.



2.2 Règles

À son tour, un joueur choisit l’un de ses trous non vides et distribue les graines une par une dans le sens anti-horaire (le trou de départ est sauté si le tour est complet).

Capture. Si la dernière graine tombe dans un trou adverse contenant alors 2 ou 3 graines, le joueur capture ces graines. La capture se propage rétroactivement tant que la condition est vérifiée.

Alimentation obligatoire. Un joueur ne peut pas jouer un coup laissant l’adversaire sans graines, sauf s’il n’a pas d’alternative.

Fin de partie. La partie se termine quand un joueur a capturé plus de 24 graines, quand un joueur ne peut plus jouer, ou par position cyclique. Le joueur ayant capturé le plus de graines gagne.

3 Représentation et architecture

Cette section décrit les structures de données internes du moteur, conçues pour maximiser les performances sous la contrainte de 100 ms par décision en moyenne en Java.

3.1 Représentation du plateau : FastBoard

Nos premières implémentations utilisaient la classe `Board` fournie par le framework, dont les méthodes (`getPlayerHoles()`, etc.) allouent de nouveaux tableaux à chaque appel. Multipliées par les millions de nœuds explorés durant la recherche, ces allocations saturent le ramasse-miettes Java (*Garbage Collector*), provoquant des pauses imprévisibles pouvant dépasser la limite de temps.

Pour résoudre ce problème, nous avons conçu une représentation interne `FastBoard` reposant sur un unique tableau primitif `int[14]`. Les indices `[0–5]` stockent les 6 trous du joueur Sud, les indices `[6–11]` ceux du joueur Nord, et les indices `[12]` et `[13]` les scores respectifs. L'état du jeu est copié et restauré par simple affectation de primitives, sans aucune allocation dynamique. Le semis, la capture et la vérification de légalité sont implémentés directement sur ce tableau, éliminant totalement les pauses du GC.

3.2 Table de Transposition

La Table de Transposition (TT) stocke les évaluations des positions déjà calculées pour éviter les recalculs redondants dans l'arbre de recherche. L'implémentation repose sur deux composants : le hachage Zobrist et une structure en tableaux parallèles.

3.2.1 Hachage Zobrist

On utilise le hachage proposé par Zobrist [4]. Une table de valeurs aléatoires 64 bits est pré-calculée à l'initialisation (50 valeurs $\times$ 12 trous). La clé de hachage d'une position est obtenue par XOR des valeurs correspondant à l'état du plateau :

$$h(s) = \bigoplus_{i=0}^{11} zobrist[state[i]][i]$$

Le hash est recalculé en $O(12)$ après chaque coup, soit un coût constant et négligeable.

3.2.2 Structure SoA (*Structure of Arrays*)

Plutôt que de stocker des objets `TTEntry` (coûteux en Java avec 16 octets d'en-tête par objet), on utilise des tableaux primitifs parallèles : `long[] hash` pour les clés Zobrist complètes (vérification de collision), `int[] score` pour les évaluations, et `byte[] depth, flag, move, gen` pour la profondeur, le type de borne, le meilleur coup et la génération. L'index dans la table est obtenu par masquage : $idx = h(s) \& (N - 1)$ où N est une puissance de 2.

3.2.3 Dimensionnement dynamique et condensation

La TT est dimensionnée différemment selon le contexte. En apprentissage, on alloue $2^{24} = 16\,777\,216$ entrées (~ 256 MiB) pour maximiser la profondeur d'analyse lors de la construction de l'*Opening Book*. En jeu, la taille est réduite à $2^{19} = 524\,288$ entrées (~ 8 MiB) pour respecter les contraintes mémoire du tournoi. Après la phase d'apprentissage, une **condensation mémoire** transfère les entrées les plus profondes et les plus récentes de la grande table vers la petite, conservant ainsi le maximum de connaissances acquises.

Un mécanisme de **vieillessement générationnel** permet d'écraser les entrées obsolètes. Chaque recherche incrémente un compteur de génération, et les entrées dont la génération est trop ancienne sont remplacées en priorité.

4 Algorithme de recherche

Cette section présente l'algorithme de recherche au cœur du moteur.

4.1 Du Minimax au Principal Variation Search

Minimax. Formalisé par von Neumann [1], cet algorithme modélise le jeu comme un arbre où Max et Min alternent. Son coût en $O(b^d)$ limite rapidement la profondeur atteignable.

Alpha-Beta. L'introduction de bornes α et β permet de couper les branches inutiles [2]. Dans le cas optimal, la complexité est réduite à $O(b^{d/2})$, doublant la profondeur.

Principal Variation Search. Le moteur utilise le PVS [3]. Le premier coup est recherché avec la fenêtre complète $[\alpha, \beta]$, les suivants avec une **fenêtre nulle** $[\alpha, \alpha + 1]$. Si un coup dépasse cette fenêtre, une re-recherche complète est lancée. Combiné à un bon ordonnancement, le PVS réduit significativement le nombre de nœuds explorés.

4.2 Approfondissement itératif et fenêtres d'aspiration

Le moteur effectue un **approfondissement itératif**. La recherche est lancée pour $d = 1, 2, \dots, d_{max}$, en s'arrêtant dès que le budget temps est presque épuisé. Un mécanisme de **time banking** répartit dynamiquement ce budget : les coups d'ouverture (Opening Book) et de fin de partie (Tablebase) consomment très peu de temps, constituant un crédit qui permet d'allouer jusqu'à 180 ms aux positions complexes du milieu de partie tout en maintenant une moyenne sous 100 ms. Des **fenêtres d'aspiration** $[\alpha_{d-1} - 25, \alpha_{d-1} + 25]$ accélèrent la convergence. En cas d'échec, la fenêtre est élargie d'un facteur 2. Pour les nœuds PV sans coup TT, un *Internal Iterative Deepening* (IID) fournit un premier coup de référence.

4.3 Techniques d'élagage

Le moteur implémente plusieurs heuristiques de coupe inspirées des moteurs d'échecs modernes. Le **Reverse Futility Pruning** coupe les sous-arbres aux nœuds non-PV de profondeur $d \leq 6$ si $eval(s) - 120d \geq \beta$ (position "trop bonne" pour l'adversaire). Le **Razoring** bascule directement en quiescence si $eval(s) + 300d \leq \alpha$ pour $d \leq 3$. Le **Late Move Pruning** ignore les coups tardifs non captants après tri, au-delà du $(2 + d)$ -ième coup pour $d \leq 4$. Le **Futility Pruning** ignore pour $d \leq 3$ un coup silencieux dont $eval(s) + 150d < \alpha$. La **Quiescence Search** explore uniquement les coups de capture à profondeur 0, avec un *Delta Pruning* (marge 300) contre l'effet d'horizon.

4.4 Late Move Reductions (LMR)

Les coups tardifs ($m \geq 3, d \geq 3$) sont explorés à profondeur réduite via :

$$R(d, m) = \max \left(0, \left\lfloor 0.5 + \frac{\ln(d) \times \ln(m)}{2.5} \right\rfloor \right)$$

La table de réductions 64×7 est pré-calculée à l'initialisation. Si le score dépasse α , une re-recherche complète confirme le résultat.

4.5 Ordonnement des coups

Un bon tri maximise les coupures précoces. Le système de priorité attribue un bonus de 10^6 au coup TT (meilleur coup de la recherche précédente), 4×10^5 et 3×10^5 aux killer moves (coups ayant provoqué une coupure β à la même profondeur), 2×10^5 au countermove (coup réfutant le dernier coup adverse), un score proportionnel à d^2 par coupure pour l'history heuristic, et un bonus K-NN par similarité avec les positions gagnantes (cf. section 5).

5 Fonction d'évaluation

La fonction d'évaluation attribue un score numérique à chaque position du plateau, guidant la recherche vers les coups les plus prometteurs. L'évaluation repose sur une combinaison linéaire de 14 caractéristiques et sur l'exploitation de la base de données fournie via K-NN.

5.1 Évaluation linéaire à 14 caractéristiques

Le score d'une position s est calculé par :

$$eval(s) = 100 \times \Delta_{captures} + \sum_{i=1}^{14} \theta_i \cdot f_i(s)$$

où $\Delta_{captures}$ est la différence de graines capturées, chaque f_i est une caractéristique extraite du plateau et θ_i son poids associé, optimisé par l'Algorithme Génétique (cf. section 6). Un bonus fixe de +200 est ajouté lorsque l'adversaire est affamé, et +5 par coup légal disponible.

Les 14 caractéristiques capturent différents aspects stratégiques de l'Awalé :

i	Caractéristique	Description
1	seedsDiff	Différence du nombre de graines en jeu entre les deux camps
2	mobilityDiff	Différence du nombre de coups légaux disponibles
3	oppVulnerable	Nombre de trous adverses contenant 1 ou 2 graines (cibles de capture)
4	myVulnerable	Nombre de nos trous vulnérables (à protéger)
5	immCaptures	Nombre de captures réalisables immédiatement
6	oppStarving	L'adversaire est affamé (2 graines ou moins sur son camp)
7	meStarving	Nous sommes affamés
8	myKroo	Nombre de nos cases contenant plus de 12 graines (<i>kroo</i>)
9	oppEmpty	Nombre de cases adverses vides
10	rightLeftBalance	Équilibre de la répartition droite/gauche de nos graines
11	capturePot3	Nombre de cases adverses contenant 1 à 3 graines (cibles de capture)
12	maxHole	Nombre de graines dans notre trou le plus chargé
13	parity	Parité du nombre total de graines capturées (influence le tempo)
14	oppBigPiles	Nombre de cases adverses contenant plus de 12 graines

Ces caractéristiques couvrent l'avantage matériel (**seedsDiff**), le contrôle territorial (**mobilityDiff**, **oppEmpty**), la tactique à court terme (**immCaptures**, **oppVulnerable**), la planification positionnelle (**myKroo**, **capturePot3**) et le tempo (**parity**).

5.2 Initialisation statistique par T-Statistic

Plutôt que d'initialiser les 14 poids aléatoirement, on exploite les 303 situations de jeu fournies. Pour chaque caractéristique f , la moyenne μ_f et l'écart-type σ_f sont calculés sur les observations gagnantes. Le poids initial est dérivé par :

$$\theta_f^{(0)} = 10 \times \left| \frac{\mu_f}{\sqrt{\max(\sigma_f^2, 0.01)}} \right|$$

Une caractéristique avec une moyenne élevée et une faible variance (signal fort et stable) obtient un poids initial important. Cette initialisation fournit un point de départ pertinent à l'Algorithme Génétique, accélérant la convergence.

5.3 K-Nearest Neighbors pour l'ordonnancement

La base de données contient 303 situations, dont 154 correspondent à des victoires. Pour exploiter ces observations en temps réel, on a implémenté un algorithme K-NN [7]. À la racine de chaque décision, la position courante est comparée aux 154 situations gagnantes par le carré de la distance euclidienne sur les 12 cases du plateau :

$$d_i = \sum_{j=1}^{12} (P_{actuel}[j] - P_{obs_i}[j])^2$$

Les $K = 15$ voisins les plus proches sont sélectionnés. Pour chaque voisin i , le coup historiquement victorieux c_i reçoit un bonus sur son score d'ordonnancement, inversement proportionnel à la distance :

$$bonus(c_i) += 50 \times \frac{1}{1 + d_i}$$

Ce mécanisme oriente la recherche vers les coups ayant statistiquement mené à des victoires dans des positions similaires, améliorant le taux de coupure Alpha-Beta sans coût de calcul significatif.

6 Apprentissage et optimisation

Le moteur exploite l'intégralité du budget de 55 minutes d'apprentissage à travers un pipeline en quatre phases.

6.1 Chargement et initialisation (< 1 min)

Les 303 observations sont chargées depuis le fichier CSV. Les 154 situations gagnantes sont extraites pour le K-NN (cf. section 5). Pour chaque caractéristique, la moyenne μ_f et l'écart-type σ_f sont calculés afin d'initialiser les poids par T-Statistic, fournissant un point de départ pertinent à l'optimisation.

6.2 Algorithme Génétique (10 minutes)

L'optimisation des 14 poids est confiée à un **Algorithme Génétique** [5]. Une population de 40 individus (vecteurs de 14 poids réels) évolue par **sélection par tournoi** ($k = 3$), **croisement uniforme** (50% de chaque parent) et **mutation gaussienne** ($\sigma = 3.0$, taux 20%). La fitness est mesurée par la somme des écarts de score sur 8 matchs simulés contre des adversaires tirés dans la population.

Le meilleur individu initial est initialisé avec les poids T-Statistic, les autres sont générés par mutation autour de cette solution. L'AG a été préféré à d'autres méthodes (Hill Climbing, SPSA) pour sa meilleure exploration de l'espace et sa robustesse face aux optima locaux.

6.3 Endgame Tablebase (15 minutes)

Pour garantir un jeu **mathématiquement parfait** en fin de partie, on construit une *Endgame Tablebase* par **analyse rétrograde** [8]. Toutes les positions terminales sont énumérées pour $n \leq 15$ graines, puis les scores optimaux sont rétro-propagés. L’indexation repose sur un schéma **combinadique** qui associe chaque distribution de graines à un index unique, évitant les collisions de hachage et réduisant la mémoire à environ 33 MiB. La base est ensuite sérialisée sur disque et accédée par *memory-mapping*, libérant le heap Java. Dès que le plateau contient 15 graines ou moins, le score exact est renvoyé instantanément.

6.4 Opening Book et condensation (~30 minutes)

Le temps restant est consacré à un **répertoire d’ouvertures** par auto-jeu profond. La TT est élargie à 16M entrées (~256 MiB). Le moteur effectue des recherches à profondeur croissante depuis la position initiale, stockant les meilleurs coups dans la TT. En compétition, les premiers coups sont joués quasi instantanément avec une profondeur inaccessible en temps réel ($d > 20$).

Dans les dix dernières minutes, les entrées les plus profondes et récentes sont **extraites** de la TT massive vers un *Opening Book* immuable séparé de la TT dynamique de jeu (512K entrées, ~8 MiB). Cette séparation garantit que l’Opening Book n’est jamais écrasé par la recherche en temps réel.

7 Résultats expérimentaux

Notre IA a été évaluée dans un championnat round-robin contre l’ensemble des bots adverses (12 adversaires, 100 matchs par paire).

Adversaire	Score obtenu	Score adverse
Bot A (Minimax)	3.00	0.00
Bot B (référence faible)	3.00	0.00
Bot C (référence faible)	3.00	0.00
Bot D (Alpha-Beta)	2.95	0.04
Bot E (PVS basique)	2.94	0.06
Bot F (PVS + Hill Climbing)	2.54	0.44
Bot G (Hybride)	2.31	0.63
Bot H (PVS + Tablebase)	2.02	0.94
Bot I (PVS + GA + TB)	1.76	1.22
Bot J (PVS + GA)	1.67	1.31
Bot K (PVS + Opening Book)	1.57	1.39
Bot L (PVS avancé)	1.50	1.47
Total	28.26	—

TABLE 1 – Résultats en tournoi : 28.26 points, bilan 12 victoires / 0 défaite.

Profondeur. En milieu de partie, le moteur atteint une profondeur de 12 à 18 demi-coups grâce au time banking (jusqu’à 180 ms pour les positions critiques). Les techniques de pruning permettent de doubler la profondeur effective par rapport à un Alpha-Beta classique.

Opening Book. Les 10–15 premiers coups sont joués instantanément grâce aux positions pré-calculées ($d > 20$).

Tablebase. En fin de partie (≤ 15 graines), le jeu est mathématiquement parfait, décisif dans les parties serrées.

8 Conclusion

La construction de cette intelligence artificielle pour le jeu d'Awalé a suivi une progression méthodique, partant d'algorithmes classiques de théorie des jeux pour aboutir à une architecture comparable, à son échelle, aux moteurs d'échecs contemporains.

Bilan. Le moteur intègre une architecture zéro-allocation (représentation `FastBoard` et Table de Transposition en Structure of Arrays) qui élimine totalement les pauses du ramasse-miettes Java, garantissant le respect de la contrainte temporelle. La recherche avancée par Principal Variation Search, enrichie de cinq techniques d'élagage (Reverse Futility, Razoring, LMP, Futility, Delta) et du Late Move Reductions, permet d'atteindre des profondeurs de 12 à 18 en milieu de partie. Le pipeline d'apprentissage exploite intégralement les 55 minutes pour l'optimisation des 14 poids par Algorithme Génétique (10 min), la construction de l'Endgame Tablebase par indexation combinadique (15 min) et la génération d'un Opening Book par auto-jeu profond (~30 min). L'algorithme K-NN appliqué aux 154 observations gagnantes guide l'ordonnancement des coups vers les motifs historiquement victorieux.

Limites. L'architecture mono-thread imposée par le règlement et le budget moyen de 100 ms par coup limitent la profondeur atteignable. L'évaluation linéaire à 14 caractéristiques, bien que performante, ne capture pas certaines structures positionnelles complexes de l'Awalé.

Perspectives. Plusieurs axes d'amélioration restent envisageables. Une première extension serait d'**enrichir le signal d'apprentissage du GA** par un feedback sur les erreurs tactiques. Au lieu d'optimiser uniquement sur les victoires, la fitness pourrait intégrer une pénalité pour les coups manifestant une perte d'évaluation brutale et un bonus pour l'exploitation des erreurs adverses. Cette approche, conceptuellement similaire au RLHF (*Reinforcement Learning from Human Feedback*) utilisé pour les modèles de langage comme ChatGPT, permettrait une convergence plus rapide en fournissant un signal plus granulaire durant les 10 minutes d'optimisation génétique. L'intégration de réseaux de neurones pour l'évaluation (approche NNUE, adoptée par Stockfish depuis 2020) constituerait également une amélioration majeure. Enfin, l'augmentation du jeu de données par génération de parties (*data augmentation*) et l'exploration du Monte-Carlo Tree Search (MCTS) comme alternative au PVS pour les positions à fort facteur de branchement ouvriraient de nouvelles voies d'optimisation.

Références

- [1] J. von Neumann, *Zur Theorie der Gesellschaftsspiele*, Mathematische Annalen, vol. 100, no. 1, pp. 295–320, 1928.
- [2] D. E. Knuth et R. W. Moore, *An Analysis of Alpha-Beta Pruning*, Artificial Intelligence, vol. 6, no. 4, pp. 293–326, 1975.
- [3] A. Reinefeld, *An Improvement to the Scout Tree-Search Algorithm*, ICCA Journal, vol. 6, no. 4, pp. 4–14, 1983.
- [4] A. L. Zobrist, *A New Hashing Method with Application for Game Playing*, University of Wisconsin, Technical Report 88, 1970.
- [5] J. H. Holland, *Adaptation in Natural and Artificial Systems*, University of Michigan Press, 1975.
- [6] J. W. Romein et H. E. Bal, *Awari is Solved*, IEEE Intelligent Systems, vol. 18, no. 6, pp. 14–16, 2003.
- [7] E. Fix et J. L. Hodges, *Discriminatory Analysis : Nonparametric Discrimination, Consistency Properties*, USAF School of Aviation Medicine, Technical Report 4, 1951.
- [8] J. Schaeffer et al., *Checkers Is Solved*, Science, vol. 317, no. 5844, pp. 1518–1522, 2007.